# SCORIFY

Phase 5: Deployment & Final Product

F2602

## 1. Product Overview

Scorify is now a fully deployed web application that scores incoming sales leads in real time. A user enters lead details into the interface, clicks one button, and gets back an instant prediction: how likely this lead is to convert and how much profit they could generate. The app is hosted on Streamlit Cloud and accessible to anyone with the link.

**Live URL:** https://scorify.streamlit.app

**GitHub:** https://github.com/ahmad19sep/scorify

The app uses two trained machine learning models running behind the scenes. XGBoost predicts conversion probability and Random Forest predicts expected profit. Both were trained on 2,458 real leads from Go2Africa's CRM system during the earlier phases of this project.

## 2. How the Application Works

The interface is split into three columns:

- **Lead Profile (left column) —** basic information about the lead: travel group type (family, partner, solo, friends), trip duration, budget level, budget range, currency, age bracket, and country.
- **Marketing Source (middle column) —** how the lead found the company: original and latest marketing source (paid social, organic search, direct traffic, etc.), device type, lead type (new enquiry, repeat, referral), source detail, booking intent, and visitor type.
- **Engagement Signals (right column) —** behavioral data: number of enquiry touchpoints, page interactions, form engagement, total visits, sessions, average pageviews, time spent on site, whether they are a returning visitor, and overall engagement score.

The user fills in whatever information is available about the lead. Any field can be left as "Unknown" if the data is not available. Once the form is filled, clicking the "Score This Lead" button runs both models and displays the results instantly.

## 3. Lead Scoring System

The model outputs a raw conversion probability between 0 and 1. We scale this to a lead score from 0 to 100 so it is easier for sales teams to understand and act on. The scaling maps the raw probability to four tiers:

| Tier | Score | Raw Probability | Recommended Action |
|---|---|---|---|
| **TOP** | 80 – 100 | > 25% | Senior rep calls within 2 hours |
| **HIGH** | 50 – 79 | 5% – 25% | Sales rep follows up within 24 hours |
| **MEDIUM** | 20 – 49 | 1% – 5% | Add to targeted email campaign |
| **LOW** | 0 – 19 | < 1% | General newsletter, no sales effort |

The results screen shows four things: the tier label (TOP/HIGH/MEDIUM/LOW), the numerical lead score out of 100, the predicted profit if the lead converts, and the expected value (profit multiplied by conversion probability). It also shows a recommended action so the sales team knows exactly what to do next.

Below the results, the app shows a breakdown of the lead's key signals — green checkmarks for strong signals (high time on site, multiple visits, strong booking intent) and warning icons for weak signals (low engagement, first-time visitor, early research stage). This helps the sales team understand why a lead scored the way it did.

## 4. Instructions for Use

**To use the hosted app:**

1. Open the link: https://scorify.streamlit.app
2. Fill in the Lead Profile on the left — select travel group, trip duration, budget, country, etc.
3. Fill in the Marketing Source in the middle — how the lead found you, their device, booking intent.
4. Adjust the Engagement Signals on the right — use the sliders for visits, sessions, time on site, etc.
5. Click the red "Score This Lead" button.
6. View the results: lead score, tier, predicted profit, expected value, and recommended action.
7. Check the Key Signals section to understand what drove the score.

**To run locally:**

8. Clone the GitHub repository.
9. Install dependencies: pip install -r requirements.txt
10. Train models: python train_and_save.py
11. Launch: streamlit run app.py
12. Open http://localhost:8501 in your browser.

## 5. Features That Impact Scores Most

Through our model training and evaluation across Phases 3 and 4, we identified which features have the biggest impact on predictions:

| # | Feature | Why it matters |
|---|---|---|
| 1 | **time_on_site** | Strongest single predictor. Leads spending 5+ minutes are significantly more likely to convert. |
| 2 | **total_visits** | More visits = more interest. A lead who comes back 10+ times is much more engaged than a one-time visitor. |
| 3 | **enquiry_touchpoints** | Number of enquiries submitted. Directly tied to profit prediction — more enquiries = bigger, more customized deals. |
| 4 | **page_interactions** | Pages the lead actively interacted with (not just viewed). Strong signal of serious research. |
| 5 | **total_sessions** | Multiple sessions means the lead keeps coming back. Each return visit increases conversion likelihood. |
| 6 | **marketing_source** | Paid search leads tend to score higher than email or social media leads. |
| 7 | **booking_intent** | "Help me book" scores higher than "Help me research". Direct measure of where they are in the buying journey. |

| | | |
|---|---|---|
| 8 | **engagement_score** | Overall engagement rating. Higher scores correlate with both conversion and profit. |

The key takeaway is that what a lead does on the website (behavioral signals) matters far more than who they are (profile data). A low-budget lead who spends 8 minutes browsing, makes 4 enquiries, and visits 10 times will score much higher than a high-budget lead who visits once and leaves.

## 6. Technical Stack

| Component | Technology |
|---|---|
| **Web Framework** | Streamlit (Python-based, no HTML/CSS needed) |
| **Hosting** | Streamlit Cloud (free tier, auto-deploys from GitHub) |
| **Conversion Model** | XGBoost Regressor ($R^2 = 0.720$) |
| **Profit Model** | Random Forest Regressor ($R^2 = 0.759$) |
| **Source Code** | GitHub repository with all code and trained models |
| **Language** | Python 3.12 |
| **Libraries** | pandas, numpy, scikit-learn, xgboost, streamlit, joblib |

The entire deployment is automated. When we push a code change to GitHub, Streamlit Cloud detects it and redeploys the app within 1-2 minutes. No manual server setup or deployment pipeline is needed.